ЛАБОРАТОРНА РОБОТА №2 МОДЕЛЮВАННЯ БЕЗПІЛОТНИХ СИСТЕМ У GAZEBO ТА ARDUPILOT SITL. СТВОРЕННЯ І ТЕСТУВАННЯ АЛГОРИТМІВ КЕРУВАННЯ, ДОСЛІДЖЕННЯ ДИНАМІКИ ПОЛЬОТУ В СИМУЛЯЦІЙНОМУ СЕРЕДОВИЩІ

Мета роботи. Метою роботи є набуття практичних навичок розгортання та запуску програмного середовища моделювання безпілотної авіаційної системи на основі Gazebo, ArduPilot SITL і ROS 2, автоматизації запуску його компонентів, виконання тестового польоту та розроблення простого алгоритму керування дроном. У результаті роботи студент повинен реалізувати політ дрона до заданої точки з подальшою автоматичною посадкою.

2.1. Програмне забезпечення

Таблиця 2.1 – Програмне забезпечення

Компонент	**Призначення**
Gazebo Sim	Моделювання безпілотного апарата, фізики польоту та зовнішнього середовища
ArduPilot SITL	Програмне моделювання польотного контролера та виконання команд керування
ROS 2	Обмін даними між алгоритмом керування, ArduPilot SITL і симуляційним середовищем
Python 3	Реалізація алгоритму керування безпілотним апаратом
Початковий програмний проєкт	Приклад автоматичного зльоту, зависання та посадки дрона
Docker або локальне середовище	Ізольований або безпосередній запуск компонентів програмного комплексу
Текстовий редактор або IDE	Перегляд, редагування та запуск програмного коду

2.2. Вимоги безпеки

Під час виконання роботи використовуються лише програмні моделі безпілотних апаратів. Попри відсутність фізичного дрона необхідно дотримуватися таких вимог:

- не запускати невідомі скрипти або контейнери без попереднього перегляду їхнього вмісту;
- не використовувати системні команди, які можуть видалити або перезаписати особисті файли;
- не змінювати системні мережеві налаштування без необхідності;
- зберігати створені файли та програмний код у робочій директорії лабораторної роботи;

- перед повторним запуском перевіряти відсутність попередніх процесів Gazebo, SITL і ROS 2, які можуть займати необхідні порти;
- не переносити алгоритм керування на реальний дрон без окремої перевірки, адаптації та дозволу викладача.

Усі польоти в межах лабораторної роботи виконуються тільки в симуляційному середовищі.

2.3. Короткі теоретичні відомості

Gazebo забезпечує моделювання фізичного середовища, конструкції дрона, сили тяжіння, зіткнень, руху та показів віртуальних сенсорів. ArduPilot SITL виконує функції програмної моделі польотного контролера: обробляє команди, стабілізує апарат і керує його рухом. ROS 2 забезпечує обмін даними між компонентами системи. Програмний вузол може отримувати інформацію про стан дрона, викликати сервіси зміни режиму, озброєння, зльоту та посадки, а також передавати команди швидкості.

У початковому прикладі дрон виконує таку послідовність:

1. Перевірка готовності системи.
2. Перехід у режим керованого польоту.
3. Arm двигунів.
4. Зліт на задану висоту.
5. Зависання протягом визначеного часу.
6. Перехід у режим посадки.
7. Завершення роботи після приземлення.

Зависання означає утримання приблизно постійного положення та висоти безпілотного апарата.

Для польоту до заданої точки програма повинна додатково:

- отримувати поточне положення дрона;
- визначати відстань і напрямок до цілі;
- формувати команди руху;
- контролювати наближення до заданої точки;
- зупинити горизонтальний рух після досягнення цілі;
- виконати автоматичну посадку.

2.4. Індивідуальне завдання

Кожному студенту викладач задає координати кінцевої точки:

$$P_{target} = (x_{target}, y_{target})$$

Після зльоту дрон повинен:

- набрати задану висоту;
- стабілізуватися в режимі зависання;
- переміститися до індивідуально заданих координат;
- зупинитися в межах допустимої похибки;
- автоматично виконати посадку;
- коректно завершити програму.

Заборонено без погодження змінювати початкове положення дрона, модель світу або задані координати кінцевої точки. Рекомендована допустима похибка досягнення цілі становить 0,2-0,3 м.

2.5. Порядок виконання лабораторної роботи

Завдання 1. Підготовка програмного середовища

- Ознайомитися зі складом наданого програмного проєкту.
- Встановити або підготувати Gazebo Sim, ArduPilot SITL, ROS 2, Python та необхідні залежності. Найкраще використовувати ОС Linux. Офіційні інструкції доступні за посиланнями: https://ardupilot.org/dev/docs/SITL-setup-landingpage.html https://ardupilot.org/dev/docs/ros2-install.html https://ardupilot.org/dev/docs/ros2-gazebo.html
- Перевірити сумісність версій програмного забезпечення.
- Запустити симуляційне середовище відповідно до інструкції для своєї операційної системи.
- Перевірити появу моделі дрона у Gazebo.
- Перевірити доступність ROS 2 вузлів, тем і сервісів, необхідних для керування дроном.

Студент повинен зафіксувати всі команди та дії, необхідні для розгортання і запуску середовища. Ця послідовність має бути наведена у звіті.

Завдання 2. Автоматизація запуску

Створити один або декілька скриптів, які автоматизують запуск компонентів симуляційного середовища.

Залежно від операційної системи можна використовувати:

- Bash-скрипти (*.sh*);
- PowerShell-скрипти (*.ps1*);
- командні файли Windows (*.bat*);
- Docker Compose;

- інший погоджений спосіб автоматизації.

Скрипт або набір скриптів повинен запускати необхідні компоненти системи без повторного ручного введення великої кількості команд. У звіті потрібно описати:

- назву й версію операційної системи;
- спосіб встановлення компонентів;
- порядок їхнього запуску;
- призначення створених скриптів;
- команди для запуску і зупинки середовища;
- відомі особливості або обмеження обраного способу.

Завдання 3. Запуск початкового прикладу

1. Запустити Gazebo, ArduPilot SITL і ROS 2.
2. Створити робочий простір ROS 2 та перейти до каталогу src.
   mkdir -p {WORKSPACE_FOLDER}/fpv_labs/src
   cd {WORKSPACE_FOLDER}/fpv_labs/src
3. Створити Python-пакет ROS 2:
   *ros2 pkg create fpv_lab2 *
   *--build-type ament_python *
   *--license Apache-2.0 *
   --dependencies rclpy std_srvs geometry_msgs nav_msgs ardupilot_msgs
4. Розмістити наданий викладачем програмний код у створеному пакеті відповідно до його структури. Перевірити, що виконуваний вузол доданий до секції console_scripts у файлі setup.py.
5. Перед складанням пакета варто явно підключити базове середовище ROS 2 та середовище ArduPilot:
   source /opt/ros/humble/setup.bash
   source /opt/ardu_ws/install/setup.bash
6. Перейти до кореневого каталогу робочого простору та виконати складання пакета:
   cd {WORKSPACE_FOLDER}/fpv_labs
   colcon build --symlink-install --packages-select fpv_lab2
7. Підключити створене робоче середовище:
   source {WORKSPACE_FOLDER}/fpv_labs/install/setup.bash
8. Перевірити доступність виконуваного вузла:
   ros2 pkg executables fpv_lab2
9. В окремому терміналі підключити системне та створене робочі середовища ROS 2. Конкретні шляхи можуть відрізнятися залежно від способу встановлення:
   source /opt/ros/humble/setup.bash

source /opt/ardu_ws/install/setup.bash
source {WORKSPACE_FOLDER}/install/setup.bash

10. Запустити наданий викладачем приклад:
    ros2 run fpv_lab2 flight_test
11. Переконатися, що дрон виконує автоматичний зліт.
12. Перевірити утримання висоти в режимі зависання.
13. Дочекатися автоматичної посадки.
14. Зберегти журнал виконання програми або його основну частину для звіту.

Початковий приклад використовується для перевірки правильності розгортання середовища. Його успішне виконання не є завершенням лабораторної роботи.

Завдання 4. Розроблення власного алгоритму

На основі наданого прикладу додати власний програмний модуль, який забезпечить політ до координат, заданих викладачем.

Алгоритм повинен:

1. Виконати перевірку готовності системи.
2. Перевести дрон у необхідний режим.
3. Перевести двигуни в режим arm.
4. Виконати зліт на задану висоту.
5. Отримувати поточне положення дрона.
6. Обчислювати відстань і напрямок до цільової точки.
7. Формувати команди горизонтального руху.
8. Зменшувати швидкість під час наближення до цілі або іншим способом забезпечувати точне позиціонування.
9. Зафіксувати досягнення точки з установленою похибкою.
10. Зупинити горизонтальний рух.
11. Виконати автоматичну посадку.
12. Коректно завершити роботу програми.

Рекомендовано реалізувати власний алгоритм як окремий клас або модуль, не змінюючи базові класи взаємодії з ROS 2 та ArduPilot без необхідності.

Завдання 5. Перевірка результату

Під час виконання польоту необхідно зафіксувати:

- початкове положення дрона;
- задані координати цілі;
- фактичні координати перед початком посадки;
- похибку досягнення цільової точки;

- факт успішного завершення посадки;
- повідомлення програми про основні етапи польоту.

Необхідно зробити щонайменше два знімки екрана:

- модель дрона на початку маршруту;
- модель дрона в кінцевій точці перед посадкою або після завершення посадки.

На знімках має бути видно симуляційне середовище та положення дрона відносно елементів світу.

2.6. Типові проблеми та способи виправлення

Таблиця 2.2 – Типові проблеми та способи виправлення

Проблема	**Імовірна причина**	**Спосіб усунення**
Модель дрона не з’являється	Gazebo не завантажив модель або ресурс	Перевірити шлях до моделей, журнали Gazebo та параметри запуску
ROS 2 не бачить вузли	Не підключене робоче середовище ROS 2	Виконати підключення setup.bash або відповідного скрипту
Сервіс недоступний	ArduPilot SITL або DDS-агент ще не запущений	Перевірити процеси, теми, сервіси та порядок запуску
Неможливо виконати arm	Не пройдено перевірку готовності або обрано неправильний режим	Переглянути повідомлення pre-arm check та стан автопілота
Дрон злітає, але не рухається	Команди надсилаються в неправильну тему або систему координат	Перевірити назву теми, тип повідомлення і frame_id
Дрон пролітає ціль	Завелика швидкість або відсутнє сповільнення	Зменшити максимальну швидкість і додати контроль відстані
Програма не завершується	Не оброблено завершення посадки	Додати перевірку висоти, стану польоту та коректне завершення ROS 2
Повторний запуск не працює	Попередні процеси залишилися активними	Завершити процеси або перезапустити симуляційне середовище

2.7. Контроль результатів

Мінімальний практичний результат:

- симуляційне середовище встановлено та запускається;
- модель дрона відображається у Gazebo;
- ROS 2 отримує дані про стан безпілотного апарата;
- створено скрипт або набір скриптів швидкого запуску;
- наданий приклад успішно виконує зліт, зависання та посадку;
- студентський модуль виконує політ до заданих координат;
- фактична кінцева позиція відповідає цілі з допустимою похибкою;
- після досягнення точки дрон автоматично приземляється;
- у програмі передбачено коректне завершення роботи.
- код завантажено у репозиторій студента на GitHub.

2.8. Вимоги до звіту

Звіт повинен містити:

- номер, назву, мету роботи, прізвище студента, групу та дату;
- назву, версію та архітектуру операційної системи;
- перелік використаного програмного забезпечення та його версій;
- інструкцію розгортання і запуску симуляційного середовища;
- опис та код створеного скрипту швидкого запуску;
- короткий опис структури наданого програмного проєкту;
- опис алгоритму польоту до заданої точки;
- задані координати цілі;
- фактичні координати дрона перед посадкою;
- обчислену похибку позиціонування;
- доданий або змінений студентом програмний код;
- журнал або основні повідомлення виконання програми;
- знімок екрана на початку маршруту;
- знімок екрана в кінцевій точці;
- короткий висновок про результати моделювання.

Похибку досягнення цілі можна визначити за формулою:

$$e = \sqrt{(x_{actual} - x_{target})^2 + (y_{actual} - y_{target})^2}$$

2.9. Чекліст студента

- Усі необхідні компоненти середовища встановлено.
- Створено та перевірено скрипт швидкого запуску.
- Початковий приклад зльоту, зависання та посадки виконано.
- Реалізовано власний політ до індивідуальної точки.
- Дрон автоматично виконав посадку.
- Програму завершено без критичних помилок.

- У звіті наведено повну послідовність розгортання і запуску.
- Задані та фактичні координати внесено до звіту.
- Обчислено похибку позиціонування.
- Додано програмний код студента.
- Додано знімки початкового та кінцевого положення дрона.
- Код завантажено у репозиторій студента на GitHub.

Додаток 1. Початковий приклад

1. Структура пакету

```
flight_test/
├── __init__.py
├── flight_state.py                 class FlightState
├── flight_test_node.py             class FlightTestNode
├── main.py                         main()
└── handlers/
    ├── __init__.py                 HANDLERS map
    ├── handler.py                  class FlightStateHandler
    ├── waiting_for_system.py       class WaitingForSystemHandler
    ├── prearm_check.py             class PrearmCheckHandler
    ├── setting_guided.py           class SettingGuidedHandler
    ├── arming.py                   class ArmingHandler
    ├── taking_off.py               class TakingOffHandler
    ├── hovering.py                 class HoveringHandler
    ├── landing.py                  class LandingHandler
    └── waiting_for_landing.py      class WaitingForLandingHandler
```

2. Код

2.1 class FlightState

flight_state.py

```python
from enum import Enum, auto

class FlightState(Enum):
    WAITING_FOR_SYSTEM = auto()
    PREARM_CHECK = auto()
    SETTING_GUIDED = auto()
    ARMING = auto()
    TAKING_OFF = auto()
    HOVERING = auto()
    LANDING = auto()
    WAITING_FOR_LANDING = auto()
    FINISHED = auto()
    ERROR = auto()
```

2.2 class FlightTestNode

flight_test_node.py

```python
from rclpy.node import Node

from nav_msgs.msg import Odometry
from std_srvs.srv import Trigger

from ardupilot_msgs.msg import Status
```

```python
from ardupilot_msgs.srv import ArmMotors, ModeSwitch, Takeoff

from .flight_state import FlightState
from .handlers import HANDLERS


class FlightTestNode(Node):

    GUIDED_MODE = 4
    LAND_MODE = 9

    TAKEOFF_ALTITUDE = 2.0
    HOVER_DURATION = 30.0

    def __init__(self):
        super().__init__("flight_test")

        self.state = FlightState.WAITING_FOR_SYSTEM
        self.request_in_progress = False

        self.status = None
        self.odometry = None
        self.hover_started_at = None

        self._handlers = HANDLERS

        self.status_subscription = self.create_subscription(
            Status,
            "/ap/v1/status",
            self.status_callback,
            10,
        )

        self.odometry_subscription = self.create_subscription(
            Odometry,
            "/odometry",
            self.odometry_callback,
            10,
        )

        self.prearm_client = self.create_client(
            Trigger,
            "/ap/v1/prearm_check",
        )

        self.mode_client = self.create_client(
            ModeSwitch,
            "/ap/v1/mode_switch",
        )

        self.arm_client = self.create_client(
            ArmMotors,
            "/ap/v1/arm_motors",
        )

        self.takeoff_client = self.create_client(
            Takeoff,
            "/ap/v1/experimental/takeoff",
        )
```

```
        self.timer = self.create_timer(0.2, self.control_loop)

        self.get_logger().info("Flight test node started")
        self.get_logger().info("Waiting for ArduPilot and ROS services...")

    def status_callback(self, message):
        self.status = message

    def odometry_callback(self, message):
        self.odometry = message

    def services_ready(self):
        return (
            self.prearm_client.service_is_ready()
            and self.mode_client.service_is_ready()
            and self.arm_client.service_is_ready()
            and self.takeoff_client.service_is_ready()
        )

    def control_loop(self):
        handler = self._handlers.get(self.state)
        if handler is not None:
            handler.handle(self)

    def call_mode_switch(self, mode, callback):
        self.request_in_progress = True

        request = ModeSwitch.Request()
        request.mode = mode

        future = self.mode_client.call_async(request)
        future.add_done_callback(callback)

        self.get_logger().info(f"Requesting flight mode {mode}")

    def fail(self, message):
        self.get_logger().error(message)
        self.state = FlightState.ERROR
```

2.3 main()

main.py

```
#!/usr/bin/env python3

import rclpy

from .flight_state import FlightState
from .flight_test_node import FlightTestNode


def main(args=None):
    rclpy.init(args=args)

    node = FlightTestNode()

    try:
```

```
        while (
            rclpy.ok()
            and node.state != FlightState.FINISHED
            and node.state != FlightState.ERROR
        ):
            rclpy.spin_once(node, timeout_sec=0.2)

    except KeyboardInterrupt:
        pass

    finally:
        node.destroy_node()

        if rclpy.ok():
            rclpy.shutdown()


if __name__ == "__main__":
    main()
```

2.4 class FlightStateHandler

handlers/handler.py

```
from abc import ABC, abstractmethod
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from ..flight_test_node import FlightTestNode


class FlightStateHandler(ABC):

    @abstractmethod
    def handle(self, node: "FlightTestNode") -> None:
        pass
```

2.5 HANDLERS map

handlers/__init__.py

```
from ..flight_state import FlightState
from .arming import ArmingHandler
from .handler import FlightStateHandler
from .hovering import HoveringHandler
from .landing import LandingHandler
from .prearm_check import PrearmCheckHandler
from .setting_guided import SettingGuidedHandler
from .taking_off import TakingOffHandler
from .waiting_for_landing import WaitingForLandingHandler
from .waiting_for_system import WaitingForSystemHandler

HANDLERS: dict[FlightState, FlightStateHandler] = {
    FlightState.WAITING_FOR_SYSTEM: WaitingForSystemHandler(),
    FlightState.PREARM_CHECK: PrearmCheckHandler(),
    FlightState.SETTING_GUIDED: SettingGuidedHandler(),
    FlightState.ARMING: ArmingHandler(),
    FlightState.TAKING_OFF: TakingOffHandler(),
    FlightState.HOVERING: HoveringHandler(),
    FlightState.LANDING: LandingHandler(),
```

```
    FlightState.WAITING_FOR_LANDING: WaitingForLandingHandler(),
}
```

2.6 class WaitingForSystemHandler

handlers/waiting_for_system.py

```
from typing import TYPE_CHECKING

from ..flight_state import FlightState
from .handler import FlightStateHandler

if TYPE_CHECKING:
    from ..flight_test_node import FlightTestNode


class WaitingForSystemHandler(FlightStateHandler):

    def handle(self, node: "FlightTestNode") -> None:
        if node.status is None:
            return

        if not node.services_ready():
            return

        node.get_logger().info("ArduPilot and all services are available")
        node.state = FlightState.PREARM_CHECK
```

2.7 class PrearmCheckHandler

handlers/prearm_check.py

```
from functools import partial
from typing import TYPE_CHECKING

from std_srvs.srv import Trigger

from ..flight_state import FlightState
from .handler import FlightStateHandler

if TYPE_CHECKING:
    from ..flight_test_node import FlightTestNode


class PrearmCheckHandler(FlightStateHandler):

    def handle(self, node: "FlightTestNode") -> None:
        if not node.request_in_progress:
            self.call_prearm_check(node)

    def call_prearm_check(self, node: "FlightTestNode") -> None:
        node.request_in_progress = True

        request = Trigger.Request()
        future = node.prearm_client.call_async(request)
        future.add_done_callback(partial(self.prearm_callback, node))

        node.get_logger().info("Running pre-arm check")
```

```
    def prearm_callback(self, node: "FlightTestNode", future) -> None:
        node.request_in_progress = False

        try:
            response = future.result()
        except Exception as error:
            node.fail(f"Pre-arm service failed: {error}")
            return

        if not response.success:
            node.fail(f"Vehicle is not armable: {response.message}")
            return

        node.get_logger().info(f"Pre-arm check passed: {response.message}")
        node.state = FlightState.SETTING_GUIDED
```

2.8 class SettingGuidedHandler

handlers/setting_guided.py

```
from functools import partial
from typing import TYPE_CHECKING

from ..flight_state import FlightState
from .handler import FlightStateHandler

if TYPE_CHECKING:
    from ..flight_test_node import FlightTestNode


class SettingGuidedHandler(FlightStateHandler):

    def handle(self, node: "FlightTestNode") -> None:
        if not node.request_in_progress:
            node.call_mode_switch(
                node.GUIDED_MODE,
                partial(self.guided_mode_callback, node),
            )

    def guided_mode_callback(self, node: "FlightTestNode", future) -> None:
        node.request_in_progress = False

        try:
            response = future.result()
        except Exception as error:
            node.fail(f"GUIDED mode request failed: {error}")
            return

        if not response.status or response.curr_mode != node.GUIDED_MODE:
            node.fail(
                f"Could not enable GUIDED mode, current mode: "
                f"{response.curr_mode}"
            )
            return

        node.get_logger().info("GUIDED mode enabled")
        node.state = FlightState.ARMING
```

2.9 class ArmingHandler

handlers/arming.py

```
from functools import partial
from typing import TYPE_CHECKING

from ardupilot_msgs.srv import ArmMotors

from ..flight_state import FlightState
from .handler import FlightStateHandler

if TYPE_CHECKING:
    from ..flight_test_node import FlightTestNode


class ArmingHandler(FlightStateHandler):

    def handle(self, node: "FlightTestNode") -> None:
        if not node.request_in_progress:
            self.call_arm(node)

    def call_arm(self, node: "FlightTestNode") -> None:
        node.request_in_progress = True

        request = ArmMotors.Request()
        request.arm = True

        future = node.arm_client.call_async(request)
        future.add_done_callback(partial(self.arm_callback, node))

        node.get_logger().info("Requesting motor arming")

    def arm_callback(self, node: "FlightTestNode", future) -> None:
        node.request_in_progress = False

        try:
            response = future.result()
        except Exception as error:
            node.fail(f"Arming service failed: {error}")
            return

        if not response.result:
            node.fail("ArduPilot rejected the arming request")
            return

        node.get_logger().info("Motors armed")
        node.state = FlightState.TAKING_OFF
```

2.10 class TakingOffHandler

handlers/taking_off.py

```
import time
from functools import partial
from typing import TYPE_CHECKING

from ardupilot_msgs.srv import Takeoff
```

```
from ..flight_state import FlightState
from .handler import FlightStateHandler

if TYPE_CHECKING:
    from ..flight_test_node import FlightTestNode


class TakingOffHandler(FlightStateHandler):

    def handle(self, node: "FlightTestNode") -> None:
        if not node.request_in_progress:
            self.call_takeoff(node)

    def call_takeoff(self, node: "FlightTestNode") -> None:
        node.request_in_progress = True

        request = Takeoff.Request()
        request.alt = float(node.TAKEOFF_ALTITUDE)

        future = node.takeoff_client.call_async(request)
        future.add_done_callback(partial(self.takeoff_callback, node))

        node.get_logger().info(
            f"Requesting takeoff to {node.TAKEOFF_ALTITUDE:.1f} m"
        )

    def takeoff_callback(self, node: "FlightTestNode", future) -> None:
        node.request_in_progress = False

        try:
            response = future.result()
        except Exception as error:
            node.fail(f"Takeoff service failed: {error}")
            return

        if not response.status:
            node.fail("ArduPilot rejected the takeoff request")
            return

        node.get_logger().info("Takeoff command accepted")
        node.get_logger().info(
            f"Hovering for {node.HOVER_DURATION:.1f} seconds"
        )

        node.hover_started_at = time.monotonic()
        node.state = FlightState.HOVERING
```

2.11 class HoveringHandler

handlers/hovering.py

```
import time
from typing import TYPE_CHECKING

from ..flight_state import FlightState
from .handler import FlightStateHandler

if TYPE_CHECKING:
    from ..flight_test_node import FlightTestNode
```

```python
class HoveringHandler(FlightStateHandler):

    def handle(self, node: "FlightTestNode") -> None:
        if node.status is None or not node.status.armed:
            node.fail("Drone disarmed unexpectedly during flight")
            return

        if node.odometry is not None:
            altitude = node.odometry.pose.pose.position.z
            node.get_logger().debug(
                f"Gazebo altitude: {altitude:.2f} m"
            )

        elapsed = time.monotonic() - node.hover_started_at

        if elapsed >= node.HOVER_DURATION:
            node.get_logger().info("Hover completed, starting landing")
            node.state = FlightState.LANDING
```

2.12 class LandingHandler

handlers/landing.py

```python
from functools import partial
from typing import TYPE_CHECKING

from ..flight_state import FlightState
from .handler import FlightStateHandler

if TYPE_CHECKING:
    from ..flight_test_node import FlightTestNode


class LandingHandler(FlightStateHandler):

    def handle(self, node: "FlightTestNode") -> None:
        if not node.request_in_progress:
            node.call_mode_switch(
                node.LAND_MODE,
                partial(self.land_mode_callback, node),
            )

    def land_mode_callback(self, node: "FlightTestNode", future) -> None:
        node.request_in_progress = False

        try:
            response = future.result()
        except Exception as error:
            node.fail(f"LAND mode request failed: {error}")
            return

        if not response.status or response.curr_mode != node.LAND_MODE:
            node.fail(
                f"Could not enable LAND mode, current mode: "
                f"{response.curr_mode}"
            )
            return
```

```
        node.get_logger().info("LAND mode enabled")
        node.state = FlightState.WAITING_FOR_LANDING
```

2.13 class WaitingForLandingHandler

handlers/waiting_for_landing.py

```
from typing import TYPE_CHECKING

from ..flight_state import FlightState
from .handler import FlightStateHandler

if TYPE_CHECKING:
    from ..flight_test_node import FlightTestNode


class WaitingForLandingHandler(FlightStateHandler):

    def handle(self, node: "FlightTestNode") -> None:
        if node.status is None:
            return

        if not node.status.flying:
            node.get_logger().info("Landing completed")

            if node.odometry is not None:
                altitude = node.odometry.pose.pose.position.z
                node.get_logger().info(
                    f"Final Gazebo altitude: {altitude:.2f} m"
                )

            node.state = FlightState.FINISHED
            node.get_logger().info("Flight test completed successfully")
```

Додаток 2. Варіанти робіт

Цільова точка задається як зміщення від фактичного початкового положення дрона. Такий підхід враховує можливе незначне відхилення початкових координат моделі в Gazebo під час різних запусків:

$$x_{target} = x_0 - \Delta x$$

$$y_{target} = y_0 - \Delta y$$

де x_0 і y_0- фактичні початкові координати дрона; Δx і Δy - зміщення, визначені відповідно до варіанта; x_{target} і y_{target} - координати цільової точки. Усі цільові точки розміщені на відстані щонайменше 4 м від старту. У стандартному світі ArduPilot maze.sdf прямі маршрути від початкового положення до заданих точок не перетинають стін, а кінцеві точки мають достатній запас від перешкод. Під час виконання цієї лабораторної роботи додавання випадкових перешкод необхідно вимкнути.

Номер варіанта для студентів групи ІР-33 відповідає порядковому номеру студента в журналі групи. Для студентів групи ІР-34 номер варіанта визначається за формулою:

$$V_{\text{ІР}-34} = V_{\text{Останній ІР}-33} + N_{\text{У журналі ІР}-34}$$

де $V_{\text{Останній ІР}-33}$ - номер останнього варіанта, призначеного студентам групи ІР-33; $N_{\text{У журналі ІР}-34}$ - порядковий номер студента в журналі групи ІР-34.

Наприклад, якщо останній варіант групи ІР-33 має номер 26, студент групи ІР-34 з номером 1 у журналі виконує варіант 27, а студент з номером 7 - варіант 33.

Таблиця 2.3 – Варіанти робіт

Варіант	Δx	Δy
1	7,0	1,4
2	6,4	-0,4
3	6,4	-0,1
4	7,6	1,1
5	7,0	-0,1
6	8,2	0,2
7	5,8	0,8
8	4,6	1,4
9	5,2	-0,1
10	4,6	0,8
11	7,6	0,8
12	7,6	0,5
13	4,0	0,2
14	6,4	0,2
15	4,6	-0,4
16	5,2	0,2
17	7,0	0,8
18	4,6	0,5
19	8,2	-0,4
20	7,6	0,2
21	7,0	-0,4
22	5,2	1,4
23	5,2	0,8
24	6,4	1,4
25	4,6	1,1
26	8,2	-0,1
27	5,8	1,1

28	7,6	-0,4
29	7,0	0,5
30	6,4	0,5
31	4,6	-0,1
32	7,6	1,4
33	4,0	0,5
34	4,0	1,1
35	8,2	1,4
36	7,0	1,1
37	4,0	-0,4
38	8,2	1,1
39	5,2	0,5
40	7,6	-0,1
41	4,6	0,2
42	6,4	0,8
43	8,2	0,8
44	4,0	1,4
45	5,8	-0,4
46	5,2	1,1
47	5,8	1,4
48	4,0	0,8
49	5,8	0,2
50	5,8	0,5
51	8,2	0,5
52	5,2	-0,4
53	4,0	-0,1
54	7,0	0,2
55	5,8	-0,1
56	6,4	1,1